Ders 4 Genişletilmiş Notu: OSI Referans Modeli, İşaretler, Kanal Kapasitesi ve Polar Kodlama

Ders 4 Genişletilmiş Notu: OSI Referans Modeli, İşaretler, Kanal Kapasitesi ve Polar Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı: Katmanlı Yapıdan İşaretlere
- 2. İki Referans Model: OSI ve TCP/IP
- 3. OSI'nin Yedi Katmanı ve Genel İlkeler
- 4. Fiziksel Katman
- 5. Veri Bağı Katmanı
- 6. Ağ Katmanı
- 7. Taşıma Katmanı
- 8. Oturum, Sunu ve Uygulama Katmanları
- 9. Diğer Ağ Modelleri ve TCP/IP Katmanları
- 10. İşaretler: Örneksel ve Sayısal
- 11. İletimi Bozan Etkenler
- 12. Veri Taşıma Kapasitesi: Nyquist ve Shannon-Hartley
- 13. İşaretlerin Kodlanması
- 14. Unipolar Kodlama
- 15. Polar Kodlama: NRZ ve RZ
- 16. Bifaz Kodlama: Manchester ve Diferansiyel Manchester

1. Dersin Planı: Katmanlı Yapıdan İşaretlere

Hoca derse geçen haftayı hatırlatarak başladı. Geçen hafta bilgisayar ağlarıyla ilgili genel bir konuşma yapılmış, ağların özellikleri ve sınıflandırılması ele alınmıştı. Bu hafta ise bir ağın özelliklerinin nasıl formüle edildiği ve ağ içinde yapılması gereken işlerin nasıl parçalara bölündüğü konuşulacaktı.

Planın iki ayağı vardı. Önce başvuru (referans) modeli olarak OSI ve katmanları genel hatlarıyla anlatılacaktı. Sonra en temel kısımdan, yani kodlamadan başlanacaktı. Hoca burada "kodlama" sözcüğünün program yazmak anlamında kullanılmadığını, işaretlerin (signals) kodlanması anlamına geldiğini özellikle belirtti.

Bu sıralamanın nedeni katmanlı yapıdır. Fiziksel katman, elimizdeki veriyi, yani 0 ve 1'lerden oluşan bitleri, hat üzerinden iletilebilecek işaretlere dönüştürmekten sorumludur. Bir taraf bitleri işaretlere çeviriyorsa, karşı taraf da işaretleri yeniden bitlere çevirmelidir. Bu yüzden işaret kavramı ve işaretlerin nasıl kodlandığı dersin temelidir.

2. İki Referans Model: OSI ve TCP/IP

2.1 OSI Bir Protokol Değildir

Günümüzde iki referans modeli vardır: OSI (Open Systems Interconnection) ve TCP/IP. Hoca iki modelin tarihçesini ve politik yanını isteyenlerin keyif için okuyabileceğini söyledi.

OSI başvuru modeli ISO tarafından ortaya konmuştur. Geçen haftaki standart türlerini hatırlatarak bunun de jure bir standart olduğunu söyledi.

> **Hocanın vurgusu:** OSI bir iletişim kuralı değildir, bir protokol de değildir. Bir başvuru modelidir. Ağ tasarımını kolaylaştıran bir model sunar.

Model şunu söyler: Şu şu katmanlar vardır, bu katmanların şu özellikleri olmalıdır, şu işlev bu katmanda, bu işlev şu katmanda yapılmalıdır. Bir ağ tasarlayacaksanız bu modele göre tasarlayabilirsiniz.

2.2 Neden Bir Standart Gerekti?

İnternet öncesi dönemde bilgisayar ağları yaygınlaşırken bir standartlaşma ihtiyacı doğdu. Hem donanım hem işletim sistemi tarafında farklı üreticiler vardı. Bu üreticilerin ürünleri bir ağda bir araya geldiğinde birlikte çalışabilmeliydi. Bunun için gereken yapının özellikleri şunlardı:

- Esnek olmalı.
- Açık olmalı.
- Birlikte çalışabilir (interoperable) olmalı.
- Kolay anlatılabilir olmalı. Hocanın anlatması, öğrencinin anlaması ve o bilgiyi başkasına aktarması kolay olmalı.

2.3 İki Modelin Ters Yönde Oluşması

Bu ihtiyaç için iki yol izlendi:

- **OSI:** ISO öncülüğünde üreticilerin ve kuruluşların bir araya gelip "şu böyle olsun, bu şöyle olsun" diye tanımlamasıyla oluştu. Yani olması gerektiği gibi önce tasarlanıp sonra uygulanmak üzere hazırlandı.
- **TCP/IP:** Zaten var olan, yürüyen ve canlı olan bir uygulamadan (implementasyon) referans modeli çıkarıldı.

  Hocanın vurgusu: İki referans modeli arasında çalışma ters yöndedir.

OSI'nin süreci tasarım açısından doğru bir süreçti: İnsanlar bir araya gelir, sorunları tespit eder, buna göre bir yapı kurar, sonra uygular. Fakat bu süreci politikayla kirletirseniz, olması gerekenden uzun tutarsanız ve baştaki niyetinizden uzak tasarım kararları alırsanız işler sarpa sarar.

Bu arada canlı ve aktif bir IP uygulaması vardı. Hayatına devam etti, yaygınlaştı. De jure bir standart değildi ama de facto bir standart hâline geldi. Referans modeli de onun üzerinden oluşturuldu.

Bugün OSI modelini doğrudan kullanan bir protokol pek yoktur. Hoca, IBM'in token ring'inin bu modele birebir olmasa da çok yakın biçimde dayandığını ve hâlâ bir yerlerde kullanıldığını söyledi. TCP/IP ise her yerde kullanılır. Hocanın yorumu şuydu: Bazen çok formal olmak, her şeyi düzgün yapmaya çalışmak sizi geciktirir. Geciktiğinizde de "atı alan Üsküdar'ı geçer."

Ama bu, TCP/IP'nin mükemmel bir referans modeli olduğu anlamına gelmez. Hocaya göre protokol yığını (protocol stack) olarak da oldukça sıkıntılı bir yapıdır. Onun o dönemdeki önemli avantajı zaten çalışıyor ve hazırda var olmasıydı.

2.4 Derslerde Neden OSI Kullanılır?

Bu derste ve sonraki Ağ Teknolojileri dersinde anlatım hep OSI başvuru modeli üzerinden yapılacak. Standartlaşma sürecindeki aksaklıklara ve politik sorunlara rağmen OSI bazı temel tasarım hedeflerini tutturmuştur. Kolay anlatılabilen bir yapıdır ve birlikte çalışabilirliği oldukça temiz biçimde sağlar. Bu yüzden bilimsel anlatımların neredeyse hepsinde OSI başvuru modeli kullanılır. Hocanın deyişiyle işin sonunda evrensel bir standarttır.

3. OSI'nin Yedi Katmanı ve Genel İlkeler

3.1 Katmanlar ve Ezber Cümlesi

OSI modelinde 7 katman vardır. Aşağıdan yukarı:

No	Türkçe	İngilizce
1	Fiziksel katman	Physical Layer
2	Veri bağı katmanı	Data Link Layer
3	Ağ katmanı	Network Layer
4	Taşıma katmanı	Transport Layer
5	Oturum katmanı	Session Layer
6	Sunu katmanı	Presentation Layer
7	Uygulama katmanı	Application Layer

> **Hocanın vurgusu:** Özellikle ağla çalışacak birinin bu katmanları zaten ezbere biliyor olması lazım. Bilgisayar mühendislerinin de bilmesinde fayda var.

Yedi katmanı akılda tutmak için bir hile vardır. Her sözcüğün baş harfi bir katmanın baş harfine denk gelen bir cümle: *Please Do Not Touch Steve's Pet Alligator*. Hoca cümlenin pek anlamlı olmadığını ama işe yaradığını söyledi: P physical, D data link, N network, T transport, S session, P presentation, A application.

3.2 Uçtan Uca Katmanlar ve Süreçler

Katmanların bir kısmı uçtan uca (end-to-end), yani uç birimden uç birime çalışır. Üstten başlayarak bunlar uygulama, sunu, oturum ve taşıma katmanlarıdır.

Burada hocanın unutulmamasını istediği bir nokta vardı. Bir bilgisayardan diğerine bilgi gönderirken haberleşmeyi bilgisayarın kendisi yapmaz. Bilgisayarınız canı sıkıldığı için gidip başka bir bilgisayarla konuşmaz. O bilgisayarda çalışan bir servis ya da uygulama, başka bir uygulamayla konuşma ihtiyacı duyar. Yani uçtan uca katmanlar, iki ayrı cihaz üzerindeki süreçlerin (process) haberleşmesini tarif eder.

Ağ katmanında ise, TCP/IP'den örnek verirsek, IP adresi yalnızca bir cihazın adresidir. Yalnız IP ile iki süreci konuşturamazsınız. Süreçlerin haberleşebilmesi için süreçlerin de adreslenebilir olması gerekir.

Soru (hocanın kendi sorusu): Ağ katmanı sadece cihazı adresliyorsa, o adrese gönderilen bilgi kimseye gitmiyor mu?

Cevap: Tabii ki gidiyor. Gönderilen bilgi o cihazdaki IP yığınına gider. IP protokolünün çalışması için de bir süreç vardır, dolayısıyla doğrudan onu adreslemiş olursunuz. Ancak herhangi bir sistem servisini ya da süreci adreslemek için bir katman daha yukarı çıkmak gerekir.

3.3 Modelde Görünmeyen Katman: Ortam Erişim Denetimi

Hoca, şekilde görünmeyen bir katmandan da sık sık bahsedileceğini söyledi: ortam erişim denetimi (Medium Access Control, MAC). Bu katman şekilde yoktur, çünkü çoğu zaman veri bağı katmanının içinde bir alt katman olarak uygulanır.

Veri bağı katmanı, verinin düğümden düğüme, yani aynı ağ içinde aktarılmasını sağlar. Örneğin evdeki yerel ağda (Local Area Network) bir Ethernet çerçevesi gönderdiğinizde ağdaki diğer cihazlar onu görür ve alır. Ancak haberleşme ortamının doğası gereği gönderim tek noktaya (unicast) ya da herkese (broadcast) olabilir. Özellikle ortamın broadcast olduğu yapılarda, ağa kimin, ne zaman, nasıl erişeceğinin belirlenmesi gerekir. Bunun için ortama erişim denetlenir. MAC katmanı veri bağı katmanının bir parçası olarak kalır.

3.4 Katmanlar Arası Arayüz ve Servis İlişkisi

Her iki katman arasında bir arayüz (interface) vardır. Örneğin şekilde 6 ile 7 arasında "6-7 arayüzü" yazar. İletişim bu arayüz üzerinden gerçekleşir.

> **Hocanın vurgusu:** n. katman, kendisinin altındaki $(n-1)$. katmanın verdiği servisleri kullanır ve kendisinin üstündeki $(n+1)$. katmana servis verir.

Hocanın örneği 3. katmandı: Ağ katmanı veri bağı katmanından servis alır, taşıma katmanına servis verir. Yukarı doğru hep servis verilir.

3.5 Katman Grupları

Hoca, Amerikalıların her şeye isim koymaya meraklı olduğunu söyleyerek katman gruplarını anlattı:

- **Kullanıcı destek katmanları (7, 6, 5):** Kullanıcıya daha yakındır.
- **Ağ destek katmanları (1, 2, 3):** Donanıma, ağın fiziksel hâline daha yakındır.
- **Taşıma katmanı (4):** Ağ destek katmanlarıyla kullanıcı destek katmanları arasında bir geçiş noktasıdır. Bu yüzden ona servis katmanı da denir.

Hoca burada bir uyarı yaptı: Buradaki "servis katmanı" adını, katmanların birbirinden aldığı servislerle karıştırmayın.

3.6 Kapsülleme (Encapsulation)

Uygulama seviyesinde bir bilgi oluşturduğumuzu ve bunu karşı tarafa göndermek istediğimizi düşünelim. Bilgi her seferinde bir alt katmana verilir: 7'den 6'ya, 6'dan 5'e, sonra 4'e, 3'e, 2'ye ve en sonunda hatta çıkar. Hatta 0 ve 1'lerden ya da oradaki işaret neyse ondan oluşan bir şeye dönüşür.

Fakat her katmanın yerine getirmesi gereken kendi işleri vardır. Bu işleri yapabilmek için uygulamadan gelen ham veriye (raw data) kendi kontrol bilgilerini eklemeleri gerekir. Bu bilgi genellikle verinin önüne eklenen bir başlık (header) olarak oluşturulur. Örneğin oturum katmanı, üstten gelen verinin önüne kendi başlığını koyar. Taşımadan ağa geçerken de ağ katmanı, taşımanın verdiği verinin önüne kendi başlığını ekler.

Bunun bir istisnası vardır: İkinci katman, yani veri bağı katmanı, 3. katmandan gelen bilginin önüne başlık eklediği gibi sonuna da bir kuyruk (trailer) ekler. Kuyruk genellikle hata denetimi için eklenir. İçinde bir sağlama değeri (checksum) bulunur.

Hoca bir önceki derste konuşulan hata denetimini hatırlattı. Hata denetimi tek bir katmanda tek başına yapılan bir işlem değildir, farklı katmanlarda yapılır. Etkili olabilmesi için de böyle yapılması gerekir.

Bilgi en alta inip hattan karşıya ulaştığında bu kez ters yönde yukarı çıkar. Her katmanda bir önceki katmanın başlığı ayıklanır ve yukarıya yalnızca o katmanla ilgili kısım aktarılır. Katman katman baktığımızda:

- 7. katmanda yalnız kullanıcının verisi vardır.
- 6. katmanda: 7'nin verisi + 6'nın başlığı.
- 5. katmanda buna 5'in başlığı eklenir.
- Aynı şekilde 4'ün, 3'ün ve 2'nin başlığı eklenir. 2'de ayrıca kuyruk da vardır.
- Karşı tarafta bu yapı açıla açıla, soyula soyula çözülür.

Yani her katmanda bir önceki katmanın bilgisi kapsüllenir (encapsulate), başka bir şeyin içine yerleştirilir. Hocanın benzetmesiyle tam matruşka bebekleri gibidir: Açtıkça içinden bir tane daha çıkar.

4. Fiziksel Katman

Fiziksel katmanın temel işi bizi iletim ortamına bağlamaktır. Eldeki veriyi, yani bitleri, iletim ortamı üzerinden aktarılabilecek hâle getirmelidir. Hoca bunun şu an kafada tam canlanmayabileceğini, işaretler konusunda bir bit diziliminin (örneğin `0100001` gibi) işarete nasıl çevrildiğini ve bu sırada başımıza neler geldiğini konuşunca netleşeceğini söyledi.

Fiziksel katmanın özellikleri bağlantının tipine ve şekline göre değişir. Bağlantı bir kablo üzerinden olabileceği gibi başka ortamlar üzerinden de olabilir. Hava bunlardan biridir. Hava dendiğinde radyo işareti de, kızılötesi (infrared) de, farklı frekanslar da işin içine girer.

Fiziksel katmanın yaptığı işler şöyle sıralandı:

- Bitleri ortama uygun işaretlere dönüştürür.
- Bu dönüşümün genlikle (amplitude) mi, frekansla mı, fazla mı ya da başka bir şeyle mi yapılacağına göre dönüşümü gerçekleştirir.
- Yönlü (directional) bir iletim söz konusuysa iletimin hangi yöne yapılacağına karar verir.
- Gerektiğinde çıkış gücü üzerinde doğrudan oynar.

5. Veri Bağı Katmanı

Veri bağı katmanı üç temel görevden sorumludur. Bunları yapabilmek için yapması gereken başka işler de vardır.

5.1 Çerçeveleme (Framing)

İlk görev, veriyi bir çerçeveye oturtmaktır. Hoca bunu iki kişinin konuşmasıyla anlattı. Konuşmanın nerede başlayıp bittiğini neyle anlarız? Bazen mimiklerimizle, bazen nefesimizle, bazen duraksamamızla. Cümle bitince durur, sonraki cümleye başlarız. Bu bir yöntemdir. Telgrafla haberleşirken de cümle sonu “stop” ile belirtilir. “Stop” için ayrı bir kod vardır: Bitti, yeniden başlıyorum.

Neden gerekir? Hatta bir bozulma olursa bilginin nereden başladığı, ne söylendiği belli olmaz. Senkronu bir kez kaçırırsanız arkası gelmez. Çerçeve iki işe yarar:

- “Ben sana şunu göndereceğim” diyebilmek için verinin sınırlarını belirler.
- Olası bir hatadan sonra bir yerden yeniden başlayabilmeyi, yeniden senkrona girebilmeyi sağlar.

5.2 Hata Denetimi (Error Control)

Bilgi, aynı ağ içinde bir noktadan diğerine hatasız aktarılabilmelidir. Bu yüzden veri bağı katmanı hata denetimi yapar. Bu denetim hata düzeltme (correction) ya da hata bulma (detection) biçiminde olabilir. Ayrıntısına sonraki derslerde bakılacak.

5.3 Akış Denetimi (Flow Control)

Aynı ağda, tek atlamada (hop) birbirine erişebilen iki düğüm düşünelim. Biri çok hızlı konuşuyor, diğeri yavaş algılıyorsa büyük sorun çıkar. Çok hızlı konuşanın söylediği üç şeyden ikisi karşı tarafta anlaşılmaz. Hoca bunu çok hızlı konuşan bir arkadaşa ya da başka dildeki bir filmde çok hızlı konuşulduğu için kaçırılan cümlelere benzetti. Veri iletiminde bu kabul edilebilir bir durum değildir.

Veri iletiminde bu durumun karşılığı tampon taşmasıdır (buffer overflow). Aldığınız veriyi kenara koymaya çalışırsınız ama siz onu işleyene kadar karşı taraf daha çok bilgi gönderir. Sonunda gelen bilgiyi atmaya (drop) başlarsınız, kullanamazsınız. Bu yüzden iki taraf arasında akış denetimi olmalıdır. Alıcı göndericiye “biraz yavaş” ya da “dur” diyebilmelidir.

Akış denetimi genellikle alıcının geri bildirimleriyle yapılır. Karşıya bilginin gidip gitmediğini, en son neyin gittiğini takip etmek için alıcı tarafın geri dönüşüne ihtiyaç vardır.

5.4 Erişim Denetimi ve İki Alt Katman

Veri bağı katmanı ayrıca erişim denetiminden de sorumludur. Paylaşımlı bir ortam varsa, ortamın kim tarafından hangi koşullarda kullanılacağı belirlenmelidir. Daha önce dendiği gibi bu iş genellikle ayrı bir ortam erişim denetimi katmanı olarak tarif edilir.

Referans modellerden uygulanan protokollerde veri bağı katmanı genellikle iki parçaya bölünerek uygulanır:

- Mantıksal bağ denetimi (Logical Link Control, LLC).
- Ortam erişim denetimi (Medium Access Control, MAC).

6. Ağ Katmanı

6.1 Veri Birimlerinin Adları

Ağ katmanına gelince artık “paket” denir. Veri bağı katmanında oluşturulan yapıya çerçeve (frame), ağ katmanında paket (packet), taşıma katmanında segment denir.

> **Hocanın vurgusu:** Her katmanda oluşturulan verinin ayrı bir adı var, aklınızda olsun: frame, packet, segment.

6.2 Neden Ayrı Bir Ağ Katmanı?

Bizim derdimiz yalnızca kendi yerel ağımızdaki cihazlarla haberleşmek değildir. Herkesi birbirine bağlamak, herkesle konuşabilmek, ağı büyütebilmek isteriz. Ama ağlar tek tip (uniform) değildir, heterojendir. Bir ağın özelliği, yapısı ve bağlantı şekli diğerinden farklı olabilir.

Bütün dünya aynı ağın içinde olsaydı ve herkese tek atlamada ulaşabilseydik, veri bağı katmanıyla herkesi birbirine bağlardık. Ama böyle değil. Bu yüzden bir kaynaktan bir hedefe, farklı ağ yapıları üzerinden paketlerin iletilmesini sağlayan bir yapıya ihtiyaç vardır.

6.3 Yönlendiriciler Neden 3. Katmanda Kalır?

Referans modeli şeklinde sağda ve solda 7 katman, ortada ise yalnız 3. seviyeye kadar çıkan cihazlar, yani yönlendiriciler (router) vardı. Hoca bunun nedenini açıkladı.

Uçtan uca haberleşmede çoğu zaman bir ağ sağlayıcısından hizmet alırız. Hocanın örneklerinde birinin bağlantısı kablonet, birininki ADSL, birininki fiber, hatta dial-up bile olabilir. Ağ sağlayıcısı bu geçişleri sağlamak için her yere cihaz koymalıdır ve bu cihazların belli yetenekleri olmalıdır. Bu cihazlar 4., 5., 6. ya da 7. seviyeye kadar çıkacak biçimde de yapılabilirdi.

Neden yapılmıyor? Hocanın deyişiyle nedeni her zaman olduğu gibi parasal. Üst seviyelere çıkan cihazları binlerce, milyonlarca adet üretmek çok daha maliyetlidir. Cihazı 3. seviyede tutar ve gömülü (embedded) cihaz olarak gerçekleştirirseniz çok daha verimli olur. Anahtarlama kapasitesi aynı kalsa bile 3. değil 4. katmana çıkmaya kalkmak çok daha maliyetlidir. Bu yüzden bu cihazlar en düşük seviyede tutulur. Özel bir cihazdan bahsetmiyorsak, ara cihazın üzerinde en fazla ağ katmanı vardır.

6.4 Devre Anahtarlama ve Paket Anahtarlama

Ağ katmanının işi, kaynaktan çıkan verinin hedefe doğru iletilmesidir. Ağ katmanı bize farklı biçimlerde hizmet sunabilir. Altta kullandığı ağın yapısına göre farklı yapılar da geliştirmiş olabilir. Ama sunduğu hizmet temelde iki yapıdan biridir: devre anahtarlama (circuit switching) ya da paket anahtarlama (packet switching). Buradaki “switching” anahtarlama anlamındadır.

Paket bir noktadan diğerine birden fazla atlamayla iletilir.

- **Paket anahtarlama:** Her atlamada, gelen her paket için o paketin nereye aktarılacağına dair bir karar verilir.
- **Devre anahtarlama:** Hoca bunu eski filmlerdeki telefon santrali örneğiyle anlattı. Telefonu açarsınız, santral “evet” der, “bana Adana’yı bağla” dersiniz, “peki” der, kapatırsınız. Santral başındaki kişi kabloyla bir hattı başka bir hatta resmen anahtarlar. Anahtarlana anahtarlana sizin numaranızdan Adana’ya kadar bir bağlantı oluşur. Bugün ise numarayı doğrudan tuşlarsınız.

Ağ katmanı da benzer bir mantıkla çalışabilir: A noktasından B noktasına paket aktaracaksa, önce A ile B arasında tam bir hat kurar ve her paket aynı yoldan gider. Ya da daha esnek bir yapı seçer.

Esnek yapının avantajı şudur: Bir topoloji değişikliğinde hayatımıza devam edebiliriz. Değişikliği fark ettiğimiz anda hızlıca toparlanma (recover) şansımız vardır. Bir kez kurduğunuz devrenin başına bir şey geldiğinde ise iş karmaşıklaşır.

6.5 Mantıksal Adresleme

Ağ katmanında kaynak ve hedef cihazlardır: sizin bilgisayarınızla karşıdaki bir sunucu ya da arkadaşınızın bilgisayarı gibi iki ana bilgisayar (host). Bunlar aynı yerel ağda olmadığı için farklı adreslere ihtiyaç vardır. Veri bağı katmanındaki adres burada artık geçerli değildir, çünkü adresler farklı ağlarda farklı biçimlerde üretilir.

Bu yüzden bizi bundan soyutlayan bir yapı gerekir. Kullanılan protokolün adresleme mekanizması kullanılır. Örneğin IP kullanılıyorsa IP adreslerinden, veri bağı katmanında Ethernet kullanılıyorsa Ethernet adreslerinden bahsedilir. Bu ikisi birbirinden ayrıdır.

6.6 Ağ Katmanının Görevleri

Hoca ağ katmanını şöyle toparladı:

- **Kaynaktan hedefe paket iletimi:** Farklı düğümler üzerinden atlayarak son noktaya erişmek.
- **Mantıksal adresleme:** Farklı ağları bir araya getirmek için bir soyutlama gerekir. Bu soyutlama mantıksal adreslemeyle çözülür.
- **Yönlendirme (routing):** Hem devre hem paket anahtarlamada yönlendirme yapılır. Devre anahtarlamada ilk bağlantı isteği oluştuğu anda yönlendirme bir kez yapılır ve uçtan uca bir hat kurulur. Paket anahtarlamada her gelen paket için ayrı bir yönlendirme kararı verilir.
- **Adres dönüşümü:** Kendi yerel ağınızdan çıktınız, elinizde bir IP adresiyle hedefin bulunduğu ağa geldiniz. O ağın içinde artık fiziksel adresi bilmeniz gerekir, çünkü son atlamayı veri bağı katmanıyla yapacaksınız. Bu yüzden ağ katmanı mantıksal adresi fiziksel adrese dönüştürmelidir.
- **Çoğullama (multiplexing):** Hoca, çoğullama yapılacaksa bunun özellikle bir eniyileme olarak ağ katmanında yapılması gerektiğini söyledi. Kaynak ile hedef arasında birden fazla bağlantı isteği varsa bunları birleştirmek ve uçta açmak ağ katmanının görevidir.

  Ders sonrası not: Hoca çoğullamayı ağ katmanının görevleri arasında saydı. Klasik anlatımda birden fazla uygulama bağlantısının tek bir ağ bağlantısı üzerinden taşınması büyük olasılıkla taşıma katmanının görevi olarak verilir. TCP/IP'de bu iş port numaralarıyla yapılır.

7. Taşıma Katmanı

7.1 Segmentlere Bölme ve Yeniden Birleştirme

Taşıma katmanı da bir üst katmana servis verir, bir alt katmandan servis alır. Ama özelliği, ağ destek ve kullanıcı destek katmanları arasındaki servis noktası olmasıdır.

Hocanın örneği: Uygulamanız 10 MB'lık bir dosya göndermek istiyor. Dosya sunu katmanından oturum katmanına, oradan taşıma katmanına gelir. Oturum katmanı taşıma katmanına "taşıyıcı değil misin, taşı" der. Ama 10 MB'lık bir yapı doğrudan taşınmaz. Taşıma katmanı bu bilgiyi parçalara, segmentlere böler. Taşıma katmanındaki veriye segment denmesi buradan, segmentasyondan (segmentation) gelir.

Bölünen parçalar karşıya aktarılır ve karşı tarafta aynı biçimde yeniden oluşturulur. Bu birleştirmeye yeniden birleştirme (reassembly) denir. Bunu yapabilmek için taşıma katmanı:

- Ayırdığı her segmente bir sıra numarası (sequence number) verir.
- Segmentlerin her birini karşıya hatasız ve temiz biçimde iletir.
- Segmentleri sıraya sokar.

7.2 Apartman Benzetmesi: Port ve Soket

Hoca, haberleşenin bilgisayarın kendisi değil, üzerinde çalışan bir uygulama olduğunu yeniden hatırlattı. O hâlde şu sorunun cevabı bulunmalıdır: Bilgiyi bu bilgisayara göndereceğim, adresi de bu, ama bu bilgisayarda kime göndereceğim?

Bir apartmana bir şey gönderdiğinizi düşünün. Apartmanın bir adresi vardır ama haberleşen apartmanlar değildir, içindeki insanlardır. İnsanların daire numarası vardır. Bir dairede birden fazla kişi de yaşayabilir, bu yüzden herkesin bir de adı vardır. Daire numarası ve isim, gönderinin orada yaşayan kime geldiğini söyler.

Ağda da benzer bir yapı vardır:

- **Ağ adresi** cihazları adreslemek için kullanılır.
- **Taşıma adresi** o cihaz üzerinde çalışan servisi, uygulamayı adreslemek için kullanılır.

Taşıma adresine hizmet erişim noktası (service access point) denir. Bu erişim noktaları çoğu zaman port olarak adlandırılır. Özellikle TCP/IP'de portlardan bahsedilir. O porta erişmek, o port üzerinden bilgi alıp göndermek için bir arayüz gerekir. Buna da soket (socket) denir.

7.3 Bağlantılı ve Bağlantısız Servis

Taşıma katmanında da iki tür servis vardır:

- **Bağlantılı (connection-oriented) servis:** Veri aktarmadan önce bir bağlantı kurulur, arada bir hat oluşturulur ve bilgi oradan taşınır. Telefon haberleşmesi bunun örneğidir.
- **Bağlantısız (connectionless) servis:** Önceden bir bağlantı kurulmaz. Posta sistemi bunun örneğidir.

Hoca bağlantısız servisi telgrafla anlattı. Bir not yazdırırsınız, not oradan oraya aktarılır, en sonunda bir postacıyla gelir. Bir sonraki telgraf bambaşka bir yoldan gidebilir, getiren postacı da başka biri olabilir. Her seferinde yol değişir. Bu benzerlikten dolayı bu tür ağlara datagram denir. TCP/IP'de taşıma katmanında çok duyulacak UDP'nin (User Datagram Protocol) adı da buradan gelir. UDP bağlantısız bir haberleşme biçimidir.

> **Hocanın vurgusu:** Biri diğerinden daha iyi değildir. İkisinin de birbirine göre avantajları ve dezavantajları vardır. Seçim, yapacağınız uygulamaya ve yapmak istediğiniz işe göre yapılır.

7.4 Alt Katman Bağlantısızken Üstte Bağlantılı Servis

Ağ katmanının uçtan uca haberleşmeyi önceden bağlantı kurarak devre anahtarlamayla ya da paket anahtarlamayla sağlayabileceğini söylemiştik. Alt katmanda nasıl bir servis verildiği, üst katmanda neyin kullanılacağını biraz belirler.

Soru (hocanın sınıfa sorusu, 00:53:24): Aşağıda bağlantısız bir yapı varken, üstte "bağlantılı çalışacağım" denebilir mi?

Cevap: Sınıftan cevap gelmeyince hoca kendisi cevapladı: Denir, hatta şu an bunu kullanıyoruz. Bu sunumun yapılmasını sağlayan şey de bu. Altta IP gerçek hayatta paket anahtarlamalı çalışır, bağlantılı çalışmaz. Ama üstte, taşıma katmanında, örneğin TCP kullanılır ve TCP bağlantılı bir yapıdır.

7.5 Taşıma Katmanının Görevleri

Hoca taşıma katmanını şöyle özetledi:

- Kaynak ile hedef arasında ileti aktarımını sağlamak.
- Hizmet erişim noktaları (port ya da soket) yardımıyla uygulamaları adreslemek.
- Üst katmandan gelen bilgiyi parçalamak ve karşıya aktarmak.
- Bunu sıralı ve hatasız biçimde yapmak.
- Karşı tarafta parçaları yeniden birleştirmek.
- Akış denetimi yapmak.

7.6 Akış Denetimi Neden İki Katmanda Var?

Soru (hocanın sınıfa sorusu, 00:55:00): Veri bağı katmanında akış denetimi vardı. Taşıma katmanında neden yine var?

Hoca önce şaka yollu "psikopatız, her şeyin yapılıp yapılmadığını kontrol etmek istiyoruz" diye bir cevap önerdi, ama daha makul bir nedeni olduğunu söyledi. Sorunun doğru-yanlış sorusu

olmadığını, amacın mantık yürütmek olduğunu belirtti. Bir öğrenci sohbet penceresinden “her katmanın kendine göre hataları olabilir, o yüzden akış denetimi olmalı” diye yazdı. Hoca bunun hata denetimi olduğunu, akış denetiminin tam olarak bu olmadığını söyledi. Akış denetimiyle kastedilen, haberleşmedeki akışın hızının denetlenmesidir.

Cevap:

- **Veri bağı katmanında** iki düğüm aynı ağdadır. Arada yalnızca ortam vardır. Gönderici ortamın hızından daha hızlı gönderemez, alıcı da veriyi o hızda alır. Fakat alıcı paketleri hatla aynı hızda işleyemeyebilir. Bu yüzden göndericinin durması, yavaşlaması, beklemesi gerekebilir. Bu, alıcıdan gelen geri bildirimle sağlanır. Hoca, akış denetiminin bağlantı kurulumundaki el sıkışmayla (handshake) hiçbir ilgisi olmadığını da ekledi.
- **Taşıma katmanında** iki ucun arasını açıp araya 3-5 ayrı ağ koyduğumuzu düşünelim. Bilgi kendi ağınızdan çıkar, başka bir ağa girer, oradan başka bir ağa geçer ve sonunda hedef düğüme ulaşır. Ağların o anki durumunu bilmezsiniz. Aradaki bir cihazda, o ağdan geçen diğer trafiğin yarattığı bir tıkanıklık (congestion), bir sıkışma olabilir. Taşıma katmanının amacı bu durumu düzenlemek, gönderimi yavaşlatmaktır.

Hoca tıkanıklığı trafiğe benzetti: Evden çıkıp ara yollardan gidersiniz, sonra bir ana yola, sonra daha büyük bir yola bağlanırsınız ve en sonunda kendinizi TEM’de arabanın içinde trafikte otururken bulursunuz. Birilerinin bu keşmekeşi engellemesi, ağdaki tıkanıklığı çözmesi gerekir.

> **Ders sonrası not:** Hocanın taşıma katmanı için anlattığı ara ağlardaki yoğunluk, büyük olasılıkla tıkanıklık denetiminin (congestion control) konusudur. TCP’de akış denetimi ile tıkanıklık denetimi ayrı mekanizmalardır. Akış denetimi uçtan uca, alıcının tamponunu korumak için yapılır. Tıkanıklık denetimi ise aradaki ağı korumak içindir. Veri bağı katmanındaki akış denetimi yalnız komşu iki düğüm arasında, atlamadan atlamaya (hop-by-hop) çalışır.

8. Oturum, Sunu ve Uygulama Katmanları

8.1 Oturum Katmanı (Session Layer)

Oturum katmanının işi, haberleşen cihazlar arasında eş zamanlamayı (senkronizasyon) sağlamaktır.

Bir haberleşme başlattığınızı ve sonra belli işleri yerine getirdiğinizi düşünün. Örneğin bir web sunucusundan bir sayfa istersiniz, sunucu cevabı yollar. Bir yere tıklarsınız, başka bir sayfaya geçersiniz, yine cevap gelir. Ya da bir yere erişmek istersiniz, sizden kullanıcı adı ve şifre istenir, bilgiyi gönderirsiniz, karşı taraf onaylayınca size bilgi göndermeye devam eder, siz de gezinmeye devam edersiniz. Bu referans modelinde bu işlerin bütünü oturum katmanının sorumluluğundadır.

Oturum katmanı ayrıca şundan da sorumludur: Bambaşka bir noktadaki bir sunucuyla aktif bir oturumunuz varken bağlantınız anlık olarak sekteye uğradı. Oturumun en az kesintiyle devam etmesi gerekir. Bunun için oturum içinde kontrol noktaları (checkpoint) oluşturulur. Bir sorun olduğunda, sorunun hemen öncesindeki sağlıklı noktaya dönülür ve oradan devam edilir.

Bağlantıyla ilgili bütün kaynakların, hem normal durumda iş bitince hem de sorun durumunda, serbest bırakılması (release) da oturum katmanının işidir. Sorun olduğunda önce toparlanıp devam etmeye çalışılır. Toparlanamıyorsa, işletim sistemlerinde de konuşulduğu gibi, düzgün kapanma (graceful shutdown) olmalıdır: Kullanılan bütün kaynaklar bırakılarak kapatılır.

8.2 Sunu Katmanı (Presentation Layer)

Hoca bu katmanın canlı bir örneğini zaman zaman yaşadığımızı, ama bunun referans modeline uygun biçimde, doğru katmanda yapılmadığını söyledi.

Sunu katmanının amacı, uygulamanın ürettiği veriyi farklı sistemler arasında uyumlu hâle getirmektir. Hoca “nasıl farklı olabilir?” sorusuna gerçek hayattan iki örnek verdi:

- **Bayt sırası:** Sizin kullandığınız mimari little endian, karşı taraftaki big endian olabilir. İki bayt ve daha uzun veri tiplerinin bellekteki sıralanması farklıdır. Bunların düzeltilmesi gerekir.

- **Karakter kodlaması:** Daha üst seviyeden bir örnek olarak, sizin kullandığınız kodlama tekniği ile karşı tarafınki farklı olabilir. Sizde "A" için belirlenen ifade karşı tarafta farklıdır. Gösterimler tek tip hâle getirilerek gerekli dönüşümler yapılmalıdır.

Sunu katmanı ayrıca verinin güvenliğiyle ilgilidir. Doğrudan sorumlu olmasa da, bu konuda bir adım atılıyorsa şifreleme (encryption) ve şifre çözme işlemleri bu katmanda yapılır.

> **Ek bilgi (derste anlatılmadı):** Little endian düzende çok baytlı bir sayının en düşük anlamlı baytı belleğin en küçük adresine yazılır, big endian düzende ise en yüksek anlamlı bayt önce gelir. Ağ protokollerinde başlık alanları büyük olasılıkla big endian düzende taşınır ve bu düzene "ağ bayt sırası" (network byte order) denir. Bu yüzden little endian bir makine başlık alanlarını gönderirken ve alırken dönüştürme yapar. Sıkıştırma (compression) da birçok kaynakta büyük olasılıkla sunu katmanının görevleri arasında sayılır.

8.3 Uygulama Katmanı (Application Layer)

Uygulama katmanı tamamen kullanıcının veri aktarımını kolaylaştırmak üzerine tasarlanmıştır. Bir arayüz gibi düşünülebilir. E-posta göndermek, dosya aktarmak, uzaktaki bir sisteme oturum açmak (remote login) gibi uygulamalar kullanıcıya bu katmanda arayüz olarak sunulur.

9. Diğer Ağ Modelleri ve TCP/IP Katmanları

9.1 Diğer Modeller

OSI dışında farklı standartlar ve ağ modelleri de vardır: SNA, DNA, IPX/SPX, AppleTalk ve TCP/IP. Bugün bunların içinde kayda değer olan tek model TCP/IP'dir.

- AppleTalk oldukça uzun süre kullanıldı.
- Novell ile birlikte IPX/SPX bir süre çokça kullanıldı.
- DNA ve SNA da zamanında bolca kullanılmış modellerdir.

Ağ Teknolojileri dersinde konuşulacak tek şey neredeyse TCP/IP olacak. TCP/IP de bir standarttır ama de facto oluşmuş bir standarttır. Üniversitelerde kullanıldığı, geliştirildiği ve zamanla internetin ortaya çıktığı yer olduğu için yaygınlaşmıştır.

9.2 OSI ile TCP/IP'nin Karşılaştırılması

OSI'de 7 katman vardır. TCP/IP'de ise toplam 4 katman vardır:

TCP/IP katmanı	Örnek protokoller	Hocanın açıklaması
Ağ erişim katmanı	Ethernet, 802.11 ailesi	Üzerinde çalışılan ağa göre değişir.
İnternet katmanı	IP, ICMP	Tek seçenek IP'dir.
Taşıma katmanı	TCP, UDP, RTP	Uygulamaya göre seçilir.
Uygulama katmanı		OSI'nin üst katmanlarını karşılar.

Hoca, OSI'de uygulama ve oturum katmanlarıyla ifade edilen kısmın TCP/IP'de yalnız uygulama katmanıyla karşılandığını söyledi.

> **Ders sonrası not:** Hoca OSI'nin uygulama ve oturum katmanlarının TCP/IP'de uygulama katmanında birleştiğini söyledi. Doğrusu, OSI'nin üç üst katmanı (oturum, sunu ve uygulama) birlikte TCP/IP'nin tek uygulama katmanına karşılık gelir.

TCP/IP'de katmanlar daha azdır ve katmanların yapısı biraz farklıdır. Hoca, OSI modelinde katmanların bir yerlere yığıldığını, üste çıktıkça içlerinin boşaldığını söyledi. TCP/IP'de ise internet katmanı modelin dar noktasıdır: Orada tek seçenek IP'dir. IP'nin üstüne çıkınca taşıma katmanı oldukça kalabalık olabilir.

Ayrıca internet katmanı ile komşu katmanlar arasında, nereye ait olduğu çok net anlaşılamayan protokoller de vardır. Bunları bir katmana koyabilirsiniz, ama zaman zaman yaptıkları iş başka bir katmana girer. Bunun nedeni TCP/IP'nin masa başında oturulup planlanmış bir model olmaması, var olan bir uygulamanın modele çevrilmiş hâli olmasıdır. Bu yüzden kendi içinde kabaca yamuklukları vardır.

Hoca bu noktada ara verdi. Ders 01:17:09'da işaretlerle devam etti.

10. İşaretler: Örneksel ve Sayısal

10.1 Neden İşaret?

Elimizde bir veri vardır ve bunu bir uçtan diğer uca, aktarım ortamının özelliklerine uygun biçimde ifade ederek aktarmamız gerekir. Hoca bunu insan iletişimiyle karşılaştırdı: Yazarken bir alfabe kullanırız. İşitme engelliler el ve kol hareketleriyle anlaşır. Görüntünün olmadığı, yalnızca sesle yapılan bir konuşmayı düşünün. Bilgisayar sistemlerinde de benzer bir şeye ihtiyaç vardır: Elimizdeki 0 ve 1'leri iletim ortamı üzerinden aktarılabilecek elektromanyetik işaretlere dönüştürmemiz gerekir.

10.2 Sürekli ve Ayrık İşaretler

İki tip işaret vardır:

- **Sürekli (continuous) işaretler:** Zaman ekseninde bir devamlılığı vardır. Bunlara analog ya da örneksel işaret denir.
- **Ayrık (discrete) işaretler:** Kesiklik, kopukluk gösterir. Bunlara sayısal (digital) işaret denir.

Hoca örneksel adının nedenine girmek üzereyken slayttaki başka bir tanımı atladığını fark edip önce onu anlattı.

10.3 Periyodik ve Periyodik Olmayan İşaretler

İşaretler, örneksel de olsa sayısal da olsa, iki gruba ayrılır:

- **Periyodik işaretler:** Kendini sürekli yineleyen işaretlerdir. Belli bir periyodu vardır, periyot sonunda baştan başlar.
- **Periyodik olmayan (aperiodic) işaretler:** Herhangi bir yinelenmesi olmayan işaretlerdir.

Slaytta periyodik işaretin solda örneksel, sağda sayısal örneği vardı. Periyodik olmayan işaretlerde de solda örneksel, sağda sayısal bir işaret vardı. Bunlarda en azından görülen kısımda bir periyot yoktu.

10.4 Basit ve Karmaşık Örneksel İşaretler, Fourier

Örneksel işaretler basit ya da karmaşık olarak sınıflandırılabilir. Basit olanlar temiz bir sinüs dalgası biçiminde devam eder. Böyle bir işareti ya da periyodik olanı sinüs veya kosinüsle ifade etme şansımız vardır. Ama karmaşık bir örneksel işareti doğrudan bir formülle ifade etmek mümkün değildir.

Bunun için işaret belli aralıklarla daha basit işaretlerin parçaları olarak düşünülür. Her parça daha basit bir işaret üzerinden formüle edilir ve parçalar birleştirilir. Bunun temel aracı Fourier dönüşümüdür. Periyodik olmayan bir işaret çok sayıda periyodik işarete ayrılır ve her birinin bu işarete etkisi bir katsayıyla ifade edilir.

Bütün işaretler bu şekilde Fourier'e açılabilir. Elinizde bu katsayılardan kaç tane varsa, bunları ters Fourier dönüşümüne sokarak aynı işareti yeniden oluşturabilirsiniz (reconstruct). Katsayılar fazlaysa ortaya orijinal işarete çok benzeyen bir şey çıkar. Azsa daha "yamuk yumuk" bir şey çıkar.

10.5 Örneksel İşaretin Özellikleri: Genlik, Frekans, Faz

Örneksel işaretlerin üç temel özelliği vardır:

- **Genlik (amplitude):** İşaretin eriştiği yüksekliktir. Hoca slayttaki işarette bunun tepe noktasıyla ifade edildiğini gösterdi. Birimi volt, amper ya da watt olabilir.
- **Frekans (sıklık):** İşaretin bir saniyede yaptığı dönüş (yineleme) sayısıdır. Birimi hertz'dir (Hz).
- **Faz:** Dalga biçiminin, örneğin bir sinüs dalgasının, nereden başlayıp nerede bittiğini ifade eden özelliktir. Hoca, periyodik olmayan işaretler de periyodik bileşenler cinsinden ifade edilebildiği için fazın onlar için de anlamlı olduğunu hatırlattı.

Frekanstan bahsederken periyottan da bahsetmek gerekir. İkisi arasında ters ilişki vardır:

$$T = \frac{1}{f}$$

Frekans hertz ile ifade edilirken periyot için saniye, milisaniye, mikrosaniye, nanosaniye ve pikosaniye gibi birimler kullanılır.

Soru (hocanın sorusu, 01:25:55): Soldaki sinüs işaretinin fazını 90 derece değiştirirseniz ne olur?

Cevap: 360 derecenin içinde 90 derece, periyodun $\frac{1}{4}$'ü kadar kayma demektir. Çeyrek periyot kaydıktan sonra işaret ilk tepe noktasından başlar, aşağıdaki tepe noktasına iner, sonra yukarıdaki tepe noktasına çıkar. Yani slayttaki sağdaki gibi olur.

Hoca ardından üç özelliğin değişimini gösteren slaytı anlattı: En üstte frekans değişince işaretin oluşma biçimi değişiyor. Ortada genlik değişince tepe noktasında fark oluşuyor. En altta faz değişiyor: İşaret aslında aynı işaret, ama başlama ve bitme noktaları kayıyor.

10.6 Sayısal İşaretin Özellikleri: Bit Hızı ve Bit Aralığı

Sayısal işaretlerde genlik, frekans ya da faz gibi özellikler yoktur. Yapı farklıdır ve genellikle periyodik olmayan özellik gösterirler. Bu yüzden bir frekanstan bahsetmek pek mümkün değildir. Sayısal işaretleri ifade etmek için iki kavram kullanılır:

- **Bit hızı (bit rate):** İlla bir ilişki kurmak gerekiyorsa, örneksel işaretlerdeki frekansla özdeş düşünülebilir.
- **Bit aralığı (bit interval):** Bir bitin aktarımı için geçen süredir, saniye cinsinden ifade edilir.

Sayısal bir işaret de yine Fourier üzerinden, farklı genlik, frekans ve faza sahip sinüs dalgalarının toplamı olarak gösterilebilir.

> **Ek bilgi (derste anlatılmadı):** İdeal bir kare dalganın Fourier serisinde büyük olasılıkla yalnız tek katlı harmonikler bulunur: temel frekans f, sonra $3f$, $5f$, $7f$ gibi bileşenler, genlikleri de harmonik numarasıyla ters orantılı olarak azalır. Kanal yalnız belli bir bant genişliğini geçirdiği için yüksek harmonikler kırpılır. Bu yüzden karşıda kenarları yuvarlanmış bir kare dalga görülür. Hocanın "katsayı azsa yamuk yumuk bir şey olur" sözü bu durumu anlatır.

11. İletimi Bozan Etkenler

11.1 Bozucu Etkenlerin Listesi

Haberleşme bir ortam üzerinden yapıldığı için ortamda haberleşmeyi bozan farklı durumlar oluşabilir:

- Zayıflama (attenuation).
- Gecikme (delay).
- Gürültü (noise).
- Çapraz girişim (crosstalk). Hoca "ses karışımı" çevirisinin bunu tam karşılamadığını söyledi.

- Bozulma (distortion).

Bu etkenler aşağı yukarı her ortamda vardır:

- **Kabloda:** Gürültü mümkündür. Çapraz girişim fazlasıyla mümkündür. Zayıflama çok olasıdır: Kabloya aktarılan işaret ilerledikçe güç kaybeder.
- **Telsiz (kablosuz) haberleşmede:** Çapraz girişim yine mümkündür. Gürültü olabilir, özellikle ortamda elektromanyetik bir gürültü varsa gönderilen işareti bozar. Zayıflama tabii ki vardır. Bunlara ek olarak yansımalar ve bozulmalar da vardır.

11.2 Çapraz Girişim Teriminin Kökeni

Hoca çapraz girişimin tam olarak "işaret karışımı" anlamına da gelmediğini söyledi. Adın geldiği yer telsizde ya da radyoda frekansların karışmasıdır. Normalde kanallar arasına koruma bantları (guard band) konur. Örneğin bir FM bandında kanallar doğrudan birbirinin üzerinde değildir, arada boşluk vardır. Fakat işaretlerde kayma (drift) olduğu için geçiş olur. 100.6'yı dinlemeniz gereken yerde bir anda 100.8'i dinlediğinizi fark edersiniz. Aynı şey telsizde de vardır. "Talk" sözcüğü geçtiği için Türkçeye ses karışımı gibi çevrilmiştir. Teknik olarak ise bir haberleşme kanalının başka bir kanalla karışması demektir.

11.3 Zayıflama ve Yükselteç

Bu sorunlar çözülemez değildir, ama çözümün ne kadar etkili olduğuna bakmak gerekir. Örneğin işaret iletim sırasında mesafeye bağlı olarak söner. Çözüm olarak araya bir yükselteç (amplifier) konabilir. Böylece işaret zayıflasa bile daha ileriye iletmeye devam edebilirsiniz.

Ancak özellikle analog işaretlerde şu sorun vardır: İşaret bir gürültüye maruz kalmışsa, onu yükselttiğinizde gürültüyü de yükseltirsiniz. Önce paraziti temizleyip sonra yükseltmek gerekir. Bu da söylendiği kadar kolay değildir.

Bir ortamda gürültü illa vardır. Hatta birinin bir şey gönderiyor olması şart değildir, hat boşken bile bir şeyler vardır. Hocanın önerdiği deney: Bir osiloskop ayarlayın ve kullanmadığınız bir kabloya bağlayın. Dümdüz bomboş bir çizgi mi görürsünüz, başka şeyler mi? Etrafta çok fazla elektromanyetik işaret vardır ve bu işaretler kablo üzerinde bir şeylerin oluşmasına neden olur.

11.4 Frekansa Bağlı Gecikme

Aynı kablo üzerinde farklı frekanslarda iletim yapabilirsiniz, örneğin 100 MHz'de de 1000 MHz'de de. Ancak işaretin frekansı değişince yayılma hızı da değişir. Bir işareti Fourier'e açtığınızda gördüğünüz bileşenler bu yüzden karşıya farklı gecikmelerle ulaşır. Hoca buna "yayılma gecikmesi" (propagation delay) dedi. Bu durum doğrudan iletim kalitesini etkiler.

> **Ders sonrası not:** Hocanın "propagation delay" dediği, farklı frekanstaki bileşenlerin farklı hızda yayılmasından doğan etkinin yaygın adı büyük olasılıkla gecikme bozulmasıdır (delay distortion). Yayılma gecikmesi ise genelde işaretin ortamda bir uçtan diğerine gitmesi için geçen süreyi anlatır.

12. Veri Taşıma Kapasitesi: Nyquist ve Shannon-Hartley

Veri taşıma kapasitesi için bilinmesi, akılda olması gereken iki teorem vardır: Nyquist teoremi ve Shannon teoremi. Hoca ikincisinin aslında Shannon-Hartley teoremi olduğunu, bir ikiliye ait olduğunu ama nedense hep "Shannon" diye söylendiğini belirtti.

12.1 Nyquist: Gürültüsüz Kanal

Nyquist teoremi en iyi durumu (optimum) anlatır. Bant genişliği H olan bir iletim ortamında, diğer uca veri aktarabilmek için bant genişliğinin iki katı örnek alınması gerektiğini söyler. Formül:

$$C = 2H \log_2 V$$

Hoca bunu örnekleme hızından (sampling rate) hatırlıyor olmamız gerektiğini, gömülü sistemler gibi derslerde konuşulmuş olabileceğini söyledi. Bir işareti kayıpsız yeniden üretmek için iki katı hızda örnekleriz: 4 MHz'lik bir işareti 8 MHz'de örnekleriz, geçişleri yakalayabilmek için. Burada da bir nevi aynı şeyden bahsedilir.

Formüldeki V, işaret (voltaj) seviyesi sayısıdır. İletim hattında 0 ve 1'lerden oluşan bir veri aktarmamız gerektiği için iki farklı voltaj seviyesi yeterlidir: 0 ve 1, -1 ve $+1$, hangisi olduğu fark etmez. İki yerine dört ya da sekiz seviye de kullanılabilir. Hoca üç ya da beş gibi tek sayıların pek kullanılmadığını söyledi. Dört seviyeniz varsa herhangi bir anda yalnızca 0 ya da 1'i değil, 00, 01, 10 ya da 11 gibi farklı kombinasyonları aktarabilirsiniz.

İkili sistemde $V = 2$ olduğundan $\log_2 2 = 1$ olur ve

$$C = 2H$$

elde edilir. Yani veri hızı bant genişliğinin en fazla iki katı olabilir.

> **Hocanın vurgusu:** Nyquist'in taşıma kapasitesi hesabında gürültü hiçbir şekilde hesaba katılmamıştır.
>
> **Ders sonrası not:** Hoca kapasite formülünü örnekleme teoremiyle birlikte anlattı. İkisi aynı kişiyle anılır ama farklı şeyler söyler. Kapasite formülü, H bant genişliğindeki gürültüsüz bir kanalda saniyede en fazla $2H$ sembol gönderilebileceğini söyler. Örnekleme teoremi ise analog bir işaretin, içindeki en yüksek frekansın en az iki katı hızda örneklenmesi gerektiğini söyler.

12.2 Çok Seviyeli İşaretin Bedeli

Gürültü hesaba katılmadığı için akla şu gelir: 0 ve 5 V yerine 0, 5 ve 10 V kullanalım. Ya da 0, 5, 10, 15 V kullanalım. Farklı işaret seviyelerine geçelim. Bunun iki sorunu vardır:

- **Geçiş süresi:** Seviyeler arasında geçiş için süre gerekir. Bu süre hızlanmanızı engeller. Tek bir işaret seviyesiyle daha fazla bilgi taşırsınız, kombinasyon artar, ama hızınız azalır.
- **Gürültüye dayanıklılık:** 0 ve 1'in güzelliği, bir şeyin ya 0 ya 1 olmasıdır. Parazit varsa değer arada bir yere çıkabilir, ama bunun için bir sınır vardır. Örneğin 5 V'luk bir sistemde 0 yerine 0,5 V gelse bile bunu hâlâ 0 kabul ederiz. 2,7 V ve üstünü 1 kabul ederiz. Bu yüzden parazite ve bozulmaya daha dayanıklıyız.

Aynı gerilim aralığında daha fazla seviye oluşturursanız, örneğin 0 ile 5 V arasında her volt için ayrı bir seviye, yani 0, 1, 2, 3, 4 ve 5 için toplam 6 seviye koyarsanız, 4 seviyesinde bir parazit olduğunda gelen değerin 5 mi 4 mü olduğunu anlamak zorlaşır. Ayrıca seviyeler arasında geçiş yaparken parazit oluşma ihtimali artar ve geçişlerde daha hızlı davranamazsınız.

12.3 İşaret-Gürültü Oranı (SNR)

Gürültüyü işin içine katmak için işaret gücünün gürültü gücüne oranı kullanılır: işaret-gürültü oranı (signal-to-noise ratio), kısaca SNR ya da S/N. Desibel cinsinden:

$$SNR_{dB} = 10 \log_{10} \frac{S}{N}$$

Birimi desibeldir (dB).

12.4 Shannon-Hartley: Gürültülü Kanal

Shannon ve Hartley, işaret-gürültü oranını veri hızı formülüne eklemenin bir yolunu bulmuştur:

$$C = H \log_2 \left(1 + \frac{S}{N}\right)$$

Buradaki $\frac{S}{N}$ desibel değil, doğrusal orandır. Bu formülle veri aktarım hızınızın en yüksek ne olabileceğini bulabilirsiniz. Hoca formülün nasıl bulunduğunu "oturmuşlar, uğraşmışlar, ölçümler de yaparak bulmuşlar" diye anlattı.

> **Ders sonrası not:** Hoca formülün ölçümlerle bulunduğunu söyledi. Formül deneysel değil kuramsaldır. Büyük olasılıkla Claude Shannon 1948'deki bilgi kuramı (information theory) çalışmasında türetmiştir. Ralph Hartley'nin büyük olasılıkla 1928'deki çalışması bunun öncülüdür.

12.5 Sayısal Örnek: 3-4 kHz Kanal, 24 dB

Soru (slayttaki soru): 3 ile 4 kHz arasında bulunan bir iletim kanalında SNR değerinin 24 dB olduğu bilindiğine göre, elde edilebilecek en yüksek hız ve iletim için kullanılabilecek ayrık seviye sayısı nedir?

Verilen:

- Kanal 3 kHz ile 4 kHz arasında.
- $SNR_{dB} = 24$ dB.

Adım 1: Bant genişliği (ara adım eklendi).

$$H = 4000 - 3000 = 1000 \text{ Hz}$$

Adım 2: Desibelden doğrusal orana. SNR'nin 24 dB olabilmesi için:

$$10 \log_{10} \frac{S}{N} = 24 \;\Rightarrow\; \log_{10} \frac{S}{N} = 2{,}4 \;\Rightarrow\; \frac{S}{N} = 10^{2{,}4} \approx 251$$

Hoca bunu "bir tarafı 10'a bölünce 2,4 olur, logaritma 10 tabanında olduğu için 10'un üssü olarak hesaplarsanız" diye anlattı.

Adım 3: Shannon-Hartley ile en yüksek hız.

$$C = 1000 \log_2(1 + 251) = 1000 \log_2 252 \approx 1000 \times 7{,}98 \approx 7978 \text{ bps} \approx 8000 \text{ bps}$$

Hoca sonucu 8000 bps olarak verdi.

Adım 4: Nyquist ile seviye sayısı. Bulunan 8000 bps'yi Nyquist formülüne koyarız:

$$8000 = 2 \times 1000 \times \log_2 V \;\Rightarrow\; \log_2 V = 4 \;\Rightarrow\; V = 2^4 = 16$$

Sonuç: En yüksek hız yaklaşık 8000 bps, gereken ayrık seviye sayısı 16'dır.

> **Ders sonrası not:** Python ile doğrulandı: $10^{2{,}4} = 251{,}19$ ve $C = 1000 \log_2(252{,}19) \approx 7978$ bps. Hoca 8000'e yuvarlayarak $V = 16$ buldu. Yuvarlamadan hesaplanırsa $\log_2 V \approx 3{,}99$ ve $V \approx 15{,}9$ çıkar. Yani 16 seviye, kapasitenin çok az üstünde bir hıza ($2 \times 1000 \times 4 = 8000$ bps) karşılık gelir. Sonuç yuvarlama sayesinde temiz çıkmaktadır.

13. İşaretlerin Kodlanması

Elimizde örneksel ve sayısal işaretler vardır ve aralarında geçiş yapabiliriz. Kodlama denince örnekselden sayısala, sayısaldan örneksele, örnekselden örneksele ve sayısaldan sayısala dönüşümler anlaşılır. Hepsi mümkündür ve hepsi konuşulacak. Hocanın örnekleri:

Dönüşüm	Örnek
Sayısal → sayısal	Bilgisayar ile çevre birimi arasındaki veri akışı.

Dönüşüm	Örnek
Örneksel → sayısal	Bir sesin örneklenip (sample) bilgisayara aktarılması.
Sayısal → örneksel	Telefon hattı üzerinden haberleşme: 0-1 bilgisi örneksele çevrilip telefon hattından ses işaretiymiş gibi gönderilir.
Örneksel → örneksel	Radyo yayını: Zaten örneksel olan ses bilgisi yine örneksel olarak yayımlanır.

Sayısal-sayısal kodlamanın üç temel çeşidi vardır:

- **Unipolar.**
- **Polar:** NRZ (non-return to zero), RZ (return to zero) ve bifaz (biphase).
- **Bipolar:** AMI, B8ZS, HDB3 gibi türleri vardır.

14. Unipolar Kodlama

Unipolar, tek bir polarite kullanılması demektir. 0 için sıfır gerilim, 1 için pozitif gerilim kullanılır. 0 için hat boşta (idle) bırakılır, hiçbir şey yapılmaz.

Hoca bu kodlamanın iki temel sorununu anlattı.

DC bileşen. Hatta her 1 geçtiğinde pozitif gerilim görülür. Bu işaretin ortalamasına baktığınızda bir DC bileşen oluşur: Ortalama genlik her zaman pozitiftir. Frekansı sıfır olan bir DC bileşen oluşur. Bu, özellikle uzun bir hat üzerinde aktarım yapıyorsanız sorun çıkarır. DC bileşen uzun hatta gitmez, zayıflar, söner. Bu yüzden işaret çok uzağa gidemez.

Eş zamanlama (senkronizasyon). İşaret bir noktadan çıkıp diğer tarafa ulaştığında iki tarafta aynı anlama gelmelidir. 1 ve 0'ı ifade etmek için belli bir süreniz vardır. Ancak veri düzgün dağılmış olmayacak, art arda gelen uzun 0 ya da 1 dizileri oluşacaktır. Geçen hafta konuşulduğu gibi, bu durumda bitin nerede başlayıp nerede bittiğini kaybedersiniz. İki sistem arasında ortak bir saat de yoktur. Unipolar kodlamanın yanına bir saat işareti koyup göndermek mümkündür. Ama aynı sorun saat için de geçerlidir: Uzun bir hatta gecikmeler yüzünden saat de kayar (drift). Birbirinden bağımsız saatler kullanılırsa onların eş zamanlanması da ayrı bir sorundur.

Sonuç olarak unipolar kodlamada iki temel sorun vardır: DC bileşen ve eş zamanlama. Bu yüzden sıkıntılı bir yapıdır.

> **Ek bilgi (derste anlatılmadı):** DC bileşenin uzun hatlarda sorun olmasının bir nedeni de, hatlarda büyük olasılıkla sık kullanılan transformatör ya da kapasitör kuplajıdır. Bu elemanlar sıfır frekanslı bileşeni geçirmez, bu yüzden ortalaması sıfır olmayan bir işaret karşı tarafta biçim değiştirir. Bu nedenle tercih edilen hat kodları ortalaması sıfıra yakın, yani DC bileşeni olmayan kodlardır.

15. Polar Kodlama: NRZ ve RZ

Unipolar kodlamada hep DC bileşen oluştuğu için alternatif yapıda polarite tek değil ikidir: Biri pozitif, biri negatif iki farklı voltaj seviyesi kullanılır. Polar kodlamanın üç alternatifi vardır: NRZ, RZ ve bifaz.

15.1 NRZ-L (Non-Return to Zero Level)

NRZ kodlamanın kendi içinde NRZ-L ve NRZ-I diye iki türü vardır. NRZ-L'de 1 dendiğinde voltaj seviyesi yukarı çıkar, 0 dendiğinde aşağı iner, yeniden 1 gelince tekrar yukarı çıkar. Yani işaret sıfırdan geçer ama sıfırda durmaz. Adındaki "sıfıra dönmeme" buradan gelir.

15.2 NRZ-I (Non-Return to Zero Inverted)

NRZ-I'de 1'ler için işaretin polaritesi değiştirilir. 0'lar için hiçbir şey yapılmaz. Hocanın slayttaki örnek üzerinden anlatımı şöyleydi: İşaret pozitifte başlamış. 0 gelince hâlâ pozitifte. 1

gelince negatife iniliyor. Bir 1 daha gelince yeniden pozitife çıkılıyor. Sonraki 0'da aynen kalınıyor. 1 gelince yine değiştiriliyor.

Soru (hocanın sorusu, 01:51:39): Bu iki kodlamada hâlâ ne problem var?

Bir öğrenci önce "işaretin bitip bitmediği fark edilmiyor mu?" diye sordu. Hoca, hiçbir şey gelmezse işaretin kesilmiş olacağını, sorunun bu olmadığını söyledi ve "art arda çok fazla sıfır gelirse ne olur?" diye yönlendirdi.

Cevap:

- **NRZ-I'de** art arda çok sayıda 0 gelirse hiçbir işaret değişimi olmaz. Sürekli aynı seviye devam eder.
- **NRZ-L'de** öğrenci önce "farklı bitler geldiğinde çok işaret değişimi var" dedi. Hoca bunu düzeltti: NRZ-L'de 1 pozitif, 0 negatif seviyedir. Dolayısıyla art arda çok 1 ya da çok 0 gelirse işaret yine hep aynı seviyede kalır.
- Her iki durumda da kayma oluşur, eş zamanlama kaybolur. Yani ikisi de sorunu doğrudan çözmez.

15.3 RZ (Return to Zero)

Eş zamanlama sorununu çözmek için RZ kodlamada üç voltaj seviyesi kullanılır: $+$, 0 ve $-$.

- 1'i ifade etmek için pozitiften başlanır ve bitin ortasında birden sıfıra inilir.
- 0'ı ifade etmek için negatiften başlanır ve bitin ortasında birden sıfıra inilir.

Böylece art arda birden fazla 0 gelse bile her bitte bir geçiş görülür: Negatifle başlar, ortada sıfıra iner, sonra yeniden negatife iner. İşaret hareketli olduğu için eş zamanlama sağlanır.

16. Bifaz Kodlama: Manchester ve Diferansiyel Manchester

Bifaz (biphase) kodlamanın iki türü vardır: Manchester ve diferansiyel Manchester.

> **Hocanın vurgusu:** Bifaz teknikleri, şu an kullanılan sayısal-sayısal kodlama teknikleri arasında en iyilerinden biridir.

16.1 RZ'nin Kalan Sorunu

Soru (hocanın sorusu, 01:54:48): RZ'de art arda çok fazla 0 gelirse nasıl bir işaretimiz olur?

Cevap: Her bit aralığında yarıya kadar negatif, sonra sıfır, sonra yine yarıya kadar negatif, sonra sıfır. Bu da beraberinde bir DC bileşen getirir: İşaretin ortalaması negatif bir gerilim olur.

Bifaz kodlamanın amacı bu DC bileşeni tamamen ortadan kaldırmaktır.

16.2 Manchester Kodlama

Manchester kodlamada:

- 1, bit ortasında negatiften pozitife geçişle ifade edilir.
- 0, bit ortasında pozitiften negatife geçişle ifade edilir.

Hoca slayttaki örneği şöyle anlattı: İlk 1'de işaret eksiden başlar, sıfırdan geçer, pozitife gelir ve orada durur. Ardından gelen 0, 1'in bıraktığı yerde (pozitifte) başlar ve negatife iner. Sonra yine 1 gelir: İşaret doğrudan eksiye iner ve yeniden artıya çıkar.

Ders sonunda bir öğrenci Manchester ile diferansiyel Manchester'ın yeniden anlatılmasını istedi. Hoca Manchester'ı şöyle özetledi: Manchester'da sabit bir kural vardır. 1, negatiften pozitife geçiş demektir. Bütün 1'lere bakın, hepsi aynıdır: Eksiden başlar, artıya çıkar. Önceki işaret ne olursa olsun böyle yapılır. Slayttaki 0, 1, 0, 1, 1 kısmında art arda gelen iki 1'de işaret artıdan devam etmez: Önce negatife iner, sonra pozitife geçer. 0'larda tam tersi, pozitiften negatife geçiş olur.

Hocanın okuduğu 0, 1, 0, 1, 1 kısmı için her bitin iki yarısı (Python ile kurala göre kontrol edildi):

Bit	0	1	0	1	1
İlk yarı	+	−	+	−	−
İkinci yarı	−	+	−	+	+

Son iki 1 arasında, bit sınırında +'dan −'ye bir ek geçiş vardır. Hocanın "art arda iki 1'de işaret önce negatife iner" sözü budur.

16.3 Diferansiyel Manchester Kodlama

Diferansiyel Manchester'da da bitin ortasında her zaman bir geçiş vardır. Bitin değerini ise bit aralığının başı belirler:

- Bit aralığının başında geçiş varsa, yani süreklilik bozuluyorsa, bu 0'dır.
- Bit aralığının başında geçiş yoksa, yani süreklilik korunuyorsa, bu 1'dir.

Hoca farkı ders sonundaki tekrarda şöyle anlattı: Manchester'daki gibi "negatiften pozitife geçiş 1, pozitiften negatife geçiş 0 olsun" denmez. Tam tersine 0 ve 1, işaretin sürekliliğini ya da süreksizliğini ifade eder. İşaretin sürekliliğini bozuyorsanız 0, bozmuyorsanız 1 ifade edersiniz.

Başlangıç için bir varsayım gerekir. Hoca, hatta hiçbir veri yokken ilk basılan 1'in, Manchester'dan gelen bir alışkanlıkla, negatiften pozitife olması gerektiğini söyledi.

Slayttaki örneğin adım adım çözümü:

- **1. bit (1):** Negatiften başlar, bit ortasında pozitife çıkar. Pozitifte biter.
- **2. bit (0):** Süreklilik bozulmalıdır. Süreklilik bozulmasaydı işaret pozitiften devam edip ortada negatife inecekti. 0 ifade edileceği için bit başında negatife inilir, ortada pozitife çıkılır. Pozitifte biter.
- **3. bit (1):** Süreklilik korunur. İşaret kaldığı yerden, yani pozitiften devam eder ve ortada negatife iner.
- **4. bit (1):** Yine kaldığı yerden, negatiften devam eder ve ortada pozitife çıkar.
- **5. bit (0):** Bit başında pozitiften negatife inilir, ortada pozitife çıkılır.
- **6. bit (1):** Süreklilik korunur. Pozitiften devam edilir ve ortada negatife inilir.

Bit	1	0	1	1	0	1
İlk yarı	−	−	+	−	−	+
İkinci yarı	+	+	−	+	+	−

Hoca dersin sonunda "yani hep aynı işareti süreklilik ya da bozulma olarak kodluyoruz" diye özetledi. Bipolar kodlamaya geçmek üzereyken dersi bitirdi.

> **Ders sonrası not:** Bu derste kullanılan Manchester kuralı (1 = negatiften pozitife geçiş) büyük olasılıkla IEEE 802.3 (Ethernet) kuralıdır. Bazı kitaplarda tersi kullanılır. Sınavda derste kullanılan kurala uymak gerekir.

Örnek Sorular

Bu derste kare yoktu. Aşağıdaki sorular derste sözlü olarak sorulan ya da slayttan sesli okunan sorulardır.

Soru 1 (derste sözlü, 00:53:24): Ağ katmanında bağlantısız (paket anahtarlamalı) bir yapı varken, üstteki katmanda bağlantılı servis verilebilir mi?

> **Çözüm (derste verildi):** Evet. Hocanın örneği: IP paket anahtarlamalı ve bağlantısız çalışır. Üstündeki TCP ise bağlantılıdır. Dersin sunulduğu sistem de bu yapı üzerinden çalışmaktadır.

Soru 2 (derste sözlü, 00:55:00): Veri bağı katmanında akış denetimi var. Taşıma katmanında neden yine akış denetimi var?

> **Çözüm (derste verildi):** Veri bağı katmanında iki düğüm aynı ağdadır ve yalnızca alıcının işleme hızına göre yavaşlamak gerekir. Taşıma katmanında ise iki uç arasında birçok farklı ağ vardır. Bu ağların herhangi birinde diğer trafiğin oluşturduğu bir tıkanıklık olabilir. Taşıma katmanı gönderimi buna göre düzenler ve yavaşlatır. Akış denetimi hata denetimiyle ya da el sıkışmayla ilgili değildir, hızla ilgilidir.

Soru 3 (derste sözlü, 01:25:55): Bir sinüs işaretinin fazı 90 derece kaydırılırsa ne olur?

> **Çözüm (derste verildi):** 90 derece, periyodun $\frac{1}{4}$'ü kadar kaymadır. Kaydırılan işaret ilk tepe noktasından başlar, aşağı tepe noktasına iner, sonra yukarı tepe noktasına çıkar.

Soru 4 (derste sözlü, 01:42:37): 3 ile 4 kHz arasında bulunan bir iletim kanalında SNR değeri 24 dB'dir. Elde edilebilecek en yüksek hız ve iletim için kullanılabilecek ayrık seviye sayısı nedir?

> **Çözüm (derste verildi):** $H = 1000$ Hz, $\frac{S}{N} = 10^{2,4} \approx 251$, $C = 1000 \log_2(252) \approx 7978 \approx 8000$ bps. Nyquist'ten $8000 = 2 \cdot 1000 \cdot \log_2 V$, buradan $V = 16$. Ayrıntı Bölüm 12.5'tedir.

Soru 5 (derste sözlü, 01:51:39): NRZ-L ve NRZ-I kodlamalarında hâlâ ne sorun vardır?

> **Çözüm (derste verildi):** NRZ-I'de art arda çok 0 gelirse işaret hiç değişmez. NRZ-L'de art arda çok 1 ya da çok 0 gelirse işaret aynı seviyede kalır. İki durumda da kayma olur ve eş zamanlama kaybolur.

Soru 6 (derste sözlü, 01:54:48): RZ kodlamada art arda çok fazla 0 gelirse işaret nasıl olur?

> **Çözüm (derste verildi):** Her bit aralığında yarıya kadar negatif, sonra sıfır olur. İşaretin ortalaması negatife kayar, yani bir DC bileşen oluşur. Bifaz kodlama bu sorunu ortadan kaldırmak için kullanılır.

Dersin Toparlaması

Bu derste önce ağda yapılması gereken işlerin katmanlara nasıl bölündüğü konuşuldu. OSI'nin bir protokol değil, ISO tarafından hazırlanmış bir başvuru modeli olduğu, TCP/IP'nin ise çalışan bir uygulamadan sonradan modele çevrildiği anlatıldı. Yedi katmanın her birinin görevi tek tek ele alındı: Fiziksel katmanın bitleri işarete çevirmesi, veri bağı katmanının çerçeveleme, hata ve akış denetimi, ağ katmanının mantıksal adresleme ve yönlendirme, taşıma katmanının segmentlere bölme, port ve soketlerle uygulamaları adresleme, oturum katmanının kontrol noktaları, sunu katmanının gösterim uyumu ve şifreleme, uygulama katmanının kullanıcıya arayüz sunması. Kapsülleme, katmanlar arası servis ilişkisi, bağlantılı ve bağlantısız servis ve akış denetiminin neden iki katmanda bulunduğu da tartışıldı.

İkinci bölümde fiziksel katmanın temeli olan işaretlere geçildi. Örneksel ve sayısal, periyodik ve periyodik olmayan işaretler, Fourier yaklaşımı, genlik, frekans, faz, bit hızı ve bit aralığı tanımlandı. İletimi bozan etkenlerden sonra Nyquist ve Shannon-Hartley teoremleriyle kanal kapasitesi hesaplandı. Son olarak sayısal-sayısal kodlamaya girildi: Unipolar kodlamanın DC bileşen ve eş zamanlama sorunları, NRZ-L, NRZ-I ve RZ'nin bu sorunları ne ölçüde çözdüğü ve bifaz kodlamanın, yani Manchester ile diferansiyel Manchester'ın, her bitte geçiş sağlayarak DC bileşeni nasıl kaldırdığı gösterildi. Bipolar kodlamaya (AMI, B8ZS, HDB3) sonraki derste geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.